实验报告

姓名：陈秀萍
学号：2026001406

实验概述

本次作业包括图像低秩近似和 MNIST 手写数字识别两个部分。第一部分使用 NumPy 的奇异值分解保留前 k 个奇异值，对灰度图像进行固定秩近似；第二部分分别使用 NumPy 和 PyTorch 训练全连接网络与卷积神经网络。正式实验使用 60000 张训练图像和 10000 张测试图像。结果表明，NumPy 与 PyTorch 的同类模型取得了接近的测试精度，卷积神经网络的测试精度约为 97.9%，高于全连接网络的约 94.7%。

1 Python 图像低秩压缩

1.1 方法

将灰度图像表示为矩阵 $A \in [0,1]^{m\times n}$，其奇异值分解为

$$A = U\,\Sigma\,V^{T}.$$

保留最大的 k 个奇异值及相应的左右奇异向量，得到

$$A_k = \sum_{i=1}^{k} \sigma_i\, u_i v_i^{T} = U_{[:,1:k]}\,\Sigma_{1:k,1:k}\,V_{[:,1:k]}^{T}.$$

程序使用 np.linalg.svd(A, full_matrices=False) 计算经济型 SVD，重建后用 np.clip 将像素截断到 $[0,1]$。函数同时检查输入矩阵非空、元素有限且位于 $[0,1]$，并要求 $1 \le k \le \min(m,n)$。

1.2 小矩阵测试

测试矩阵为 diag(0.9,0.5,0.2)，取 $k = 2$。相同输入下的 RcppEigen 参考结果为 diag(0.9,0.5,0)。NumPy 结果与参考结果的最大绝对误差为 0，输出尺寸与输入一致，所有元素均在 $[0,1]$ 内。

1.3 图像结果

原图尺寸为 1365×2048，按比例缩小后得到 341×512 的灰度矩阵，共 174592 个像素。相对 Frobenius 误差定义为

$$E_k = \frac{\| A - \widehat{A}_k \|_F}{\| A \|_F}.$$

截断 SVD 需要保存 U_k、Σ_k 和 V_k，相应数值个数为 $k(m+n+1)$。表 1 汇总了三种秩的误差和存储量。

k	相对 Frobenius 误差	低秩因子数	原图像素数	存储比例
5	0.32358	4270	174592	2.4457%
20	0.21570	17080	174592	9.7828%
50	0.13295	42700	174592	24.4570%

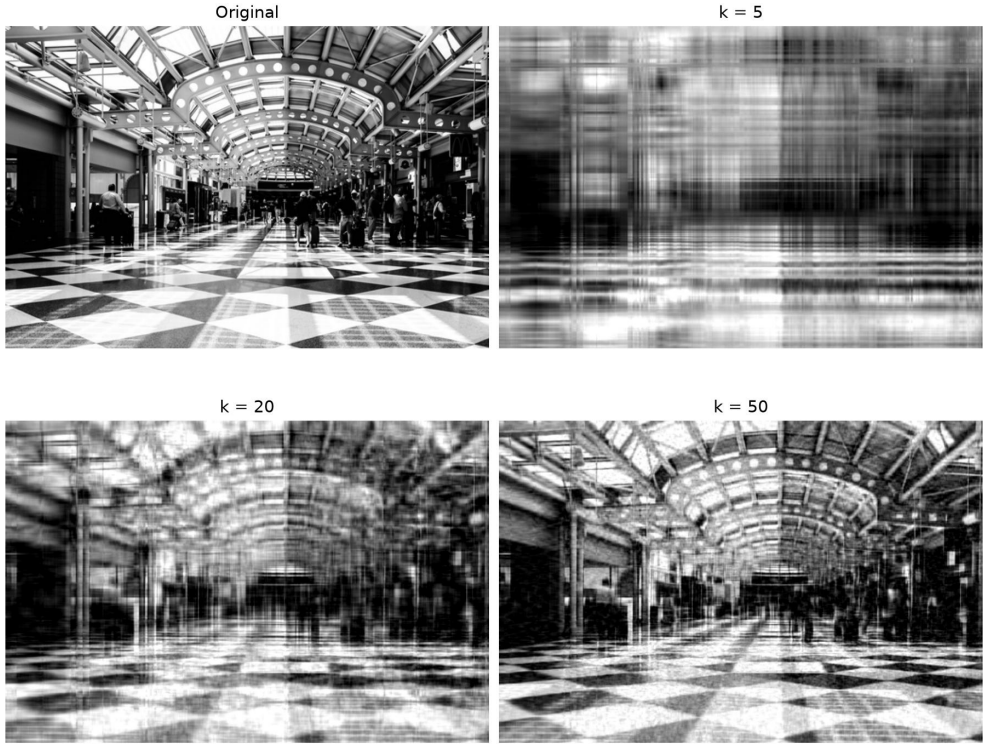


图 1 原图与 $k = 5,20,50$ 的低秩重建结果

随着 k 增大，主要轮廓、拱顶结构和地面纹理逐渐清晰，相对误差由 0.32358 降至 0.13295；相应存储比例则由 2.45% 增加到 24.46%。因此，较小的 k 节省存储但损失较多细节，较大的 k 能提高重建质量，但需要保存更多低秩因子。

2 MNIST 手写数字识别

2.1 数据与网络结构

MNIST 包含 60000 张训练图像和 10000 张测试图像，每张图像为 28×28 的单通道灰度图。像素缩放到 $[0,1]$，测试集只用于最终评价，不参与参数更新。

两种网络结构如下。

模型	输入	隐藏结构	输出
TwoLayerNet	784 维展开向量	50 个 Sigmoid 隐藏单元	10 类 logits
SimpleConvNet	$N \times 1 \times 28 \times 28$	30 个 5×5 卷积核，ReLU，2×2 最大池	10 类 logits

模型	输入	隐藏结构	输出
		化，100 个 ReLU 全连接单元	

正式训练的批大小均为 100。TwoLayerNet 使用 SGD，学习率为 0.1，共更新 10000 次；SimpleConvNet 使用 Adam，学习率为 0.001，训练 2 个 epoch，共更新 1200 次。为控制纯 NumPy CNN 的运行时间，CNN 训练轮数由题目建议的 20 个 epoch 调整为 2 个 epoch，NumPy 和 PyTorch 使用相同训练轮数。

2.2 NumPy 实现

全连接网络将每张图像展开为 784 维向量。每次更新先使用固定随机数生成器抽取 100 个训练样本，调用 gradient() 完成前向传播和反向传播，再由 SGD 更新权重与偏置。正式训练没有使用数值微分逐次求梯度。

为检查反向传播，程序在一个输入维数为 4、隐藏层为 3、输出为 2 的小网络上比较中心差分数值梯度和反向传播梯度。最大绝对差异为 1.111×10^{-11}，说明反向传播实现与数值梯度一致。

NumPy CNN 使用 im2col 将局部卷积窗口展开为矩阵乘法，并显式实现卷积层、ReLU、最大池化、全连接层和 Softmax 交叉熵损失的前向与反向传播。训练时按 epoch 打乱训练样本，然后顺序提取 mini-batch，并使用 Adam 更新参数。

2.3 PyTorch 实现

PyTorch 版本使用 nn.Linear、nn.Conv2d、nn.ReLU 和 nn.MaxPool2d 构建相同网络，权重按照均值为 0、标准差为 0.01 的正态分布初始化，偏置初始化为 0。模型直接输出 logits，并传入 nn.CrossEntropyLoss，损失函数之前没有额外使用 Softmax。

训练过程中依次执行清空梯度、前向计算、损失计算、backward() 和优化器更新。测试时调用 model.eval()，并在 torch.no_grad() 环境中进行预测。PyTorch 自动构建计算图并计算梯度，因此不需要逐层手写反向传播。

2.4 训练结果

表 2 给出四种实现的正式实验结果。

框架	模型	测试精度	训练时间 秒	更新次数	设备
NumPy	TwoLayerNet	94.78%	5.689	10000	CPU
PyTorch	TwoLayerNet	94.65%	8.450	10000	MPS
NumPy	SimpleConvNet	97.89%	50.784	1200	CPU
PyTorch	SimpleConvNet	97.93%	4.827	1200	MPS

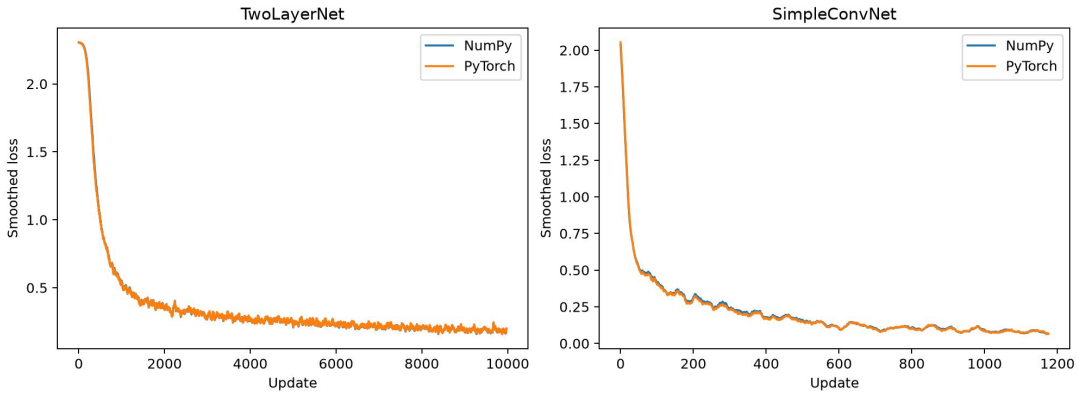


图 2 NumPy 与 PyTorch 的训练损失曲线

两种框架的同类模型具有非常接近的损失曲线和测试精度，说明网络结构、预处理和训练设置保持一致。CNN 在两个框架下均达到约 97.9% 的测试精度，比全连接网络高约 3 个百分点。CNN 保留了图像的空间结构，卷积核能够提取笔画边缘和局部形状，池化则提高了对局部位置变化的稳定性。

TwoLayerNet 的 PyTorch MPS 训练时间略长于 NumPy CPU。该网络规模较小，MPS 的数据传输和调度开销所占比例较高。对于计算量更大的 CNN，PyTorch MPS 的训练时间为 4.827 秒，明显短于 NumPy CPU 的 50.784 秒。由于两者使用不同设备，这一差异只能说明本次实际运行环境下的耗时，不能完全归因于 NumPy 与 PyTorch 框架本身。

2.5 误分类样本

下列图像展示了四种实现各自最先出现的 10 个误分类测试样本，标题同时给出真实标签和预测标签。部分数字书写较潦草，形状介于多个类别之间，例如 4 与 6、5 与 6、7 与 9，容易造成混淆。

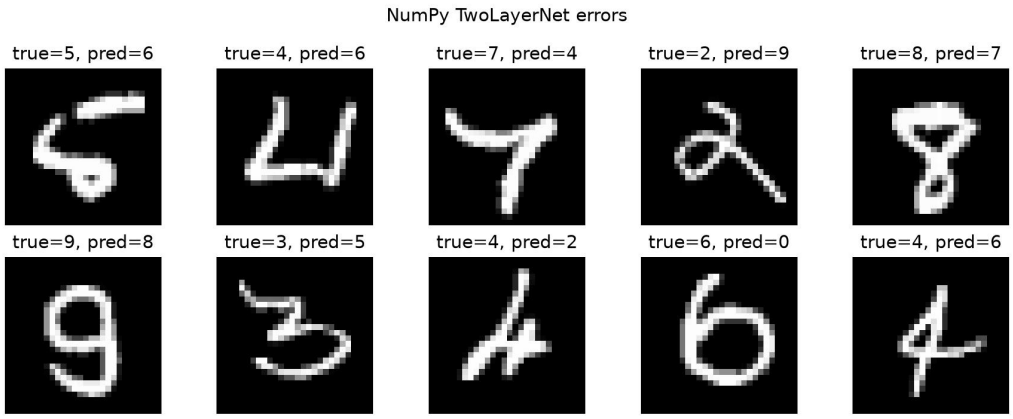


图 3 NumPy TwoLayerNet 误分类样本

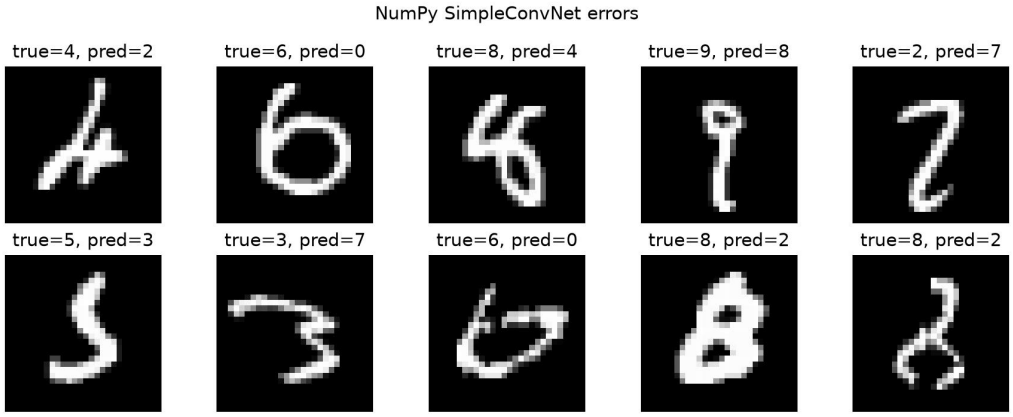


图 4 NumPy SimpleConvNet 误分类样本

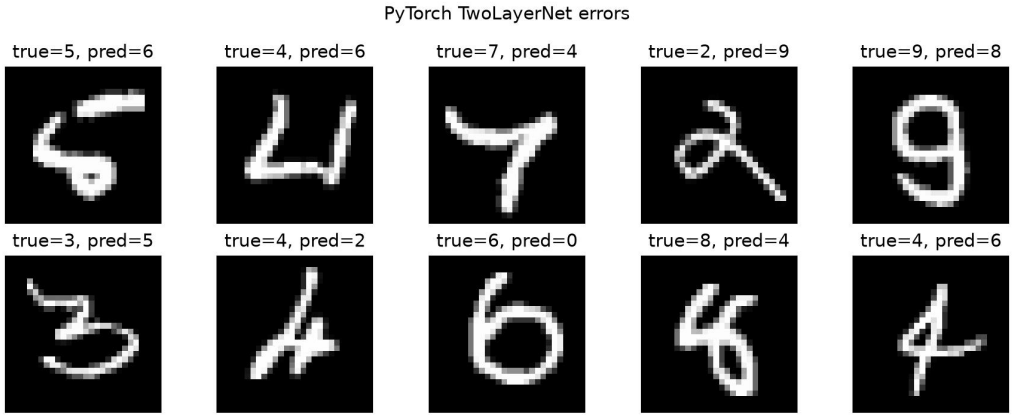


图 5 PyTorch TwoLayerNet 误分类样本

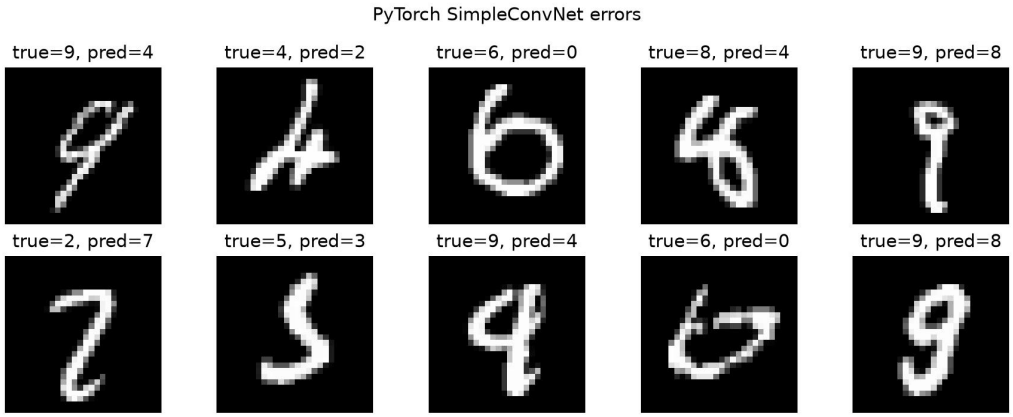


图 6 PyTorch SimpleConvNet 误分类样本

3 运行环境与复现

本次实验在 MacBook Air（Apple Silicon）上完成。Python 版本为 3.12.14，NumPy 版本为 2.5.3，PyTorch 版本为 2.15.0。NumPy 使用 CPU，PyTorch 使用 MPS。训练计时包含批次抽样、前向计算、反向传播、参数更新和损失记录，不包含数据下载与读取、测试评价、绘图和文件保存；MPS 计时前后执行同步。

程序从 MNIST 原始 IDX 压缩文件读取数据。运行 python mnist-numpy.py 和 python mnist-pytorch.py --device auto 可以复现正式训练，运行 python summarize-results.py 可以生成比较表和总损失曲线。

4 课程代码来源与验证

NumPy 网络的类接口、参数命名、层式组织和训练流程参考课程讲义“神经网络的学习与卷积神经网络”以及配套的 ch04/two_layer_net.py、ch04/train_neuralnet.py、ch07/simple_convnet.py 和 ch07/train_convnet.py。本作业按照题目要求整理并实现 NumPy 网络，并另外完成 MNIST IDX 文件读取、训练计时、梯度检查、结果绘图、PyTorch 对照实现和结果汇总。

结果验证包括：第一题的小矩阵与 RcppEigen 参考结果核对；NumPy TwoLayerNet 的数值梯度检查；NumPy CNN 的前后向传播和梯度形状检查；以及四种 MNIST 实现的损失曲线、测试精度和误分类样本检查。